

Experiments on Multithreading

1. **Title:** Demonstrate use of Multithreading concepts.

2. **Objectives:**

- **Understand** the Java Thread Model and the role of the main thread.
- **Learn** how to create and start a thread in Java.
- **Understand** how to create and execute multiple threads concurrently.
- **Learn** basic thread execution and scheduling concepts.

3. **Outcomes:**

Students will be able to

- **Explain** the Java Thread Model and main thread.
- **Create** threads using Thread and Runnable.
- **Create** and **execute** multiple threads in a Java program.
- **Understand** concurrent execution and basic thread behavior.

4. **Software/Hardware/Other Requirements:**

- Any CPU with Pentium Processor / similar, 256 MB RAM/ more, 1GB HDD / more.
- Operating System – ubuntu/Fedora 64bit OS
- Software: JDK 17 or later, Code Editor

5. **Theory:**

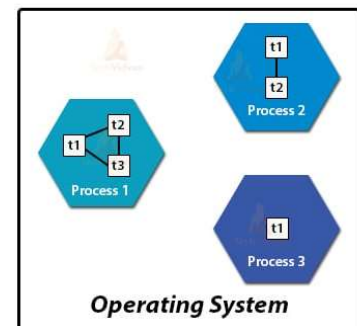
A. **What is Multithreading?**

- A thread is the smallest unit of execution within a process. It is a lightweight sub-process that can run independently while sharing the same memory space with other threads of the same process.
- Java supports multithreading, allowing multiple threads to execute concurrently within a program.

• **Key Points**

- A process is an independent program in execution.
- A thread is a lightweight part of a process.
- Multiple threads can exist within a single process.
- Threads share common resources such as heap memory but maintain their own:
 - Program Counter (PC)
 - Stack
 - Registers
- Multithreading improves responsiveness and CPU utilization.
- The process of running more than one thread in a program or an application concurrently or simultaneously is called Multithreading in Java. When more than two threads run concurrently, it is multithreading.
- Therefore, we can also call it Concurrency in Java. Multi-threading allows multiple activities to execute concurrently in the same program so as to achieve the maximum utilization of CPU.
- **Note:** On a **single-core CPU**, only one thread executes at a time and the CPU rapidly switches between threads (context switching). On a **multi-core CPU**, multiple threads can execute simultaneously on different cores
- Imagine Google Chrome is process then:
 - One thread handles the UI(A).
 - Another thread handles network requests(B).
 - Another thread renders web pages(C).
- All these threads run concurrently within the same process.

Execution of a Java Thread

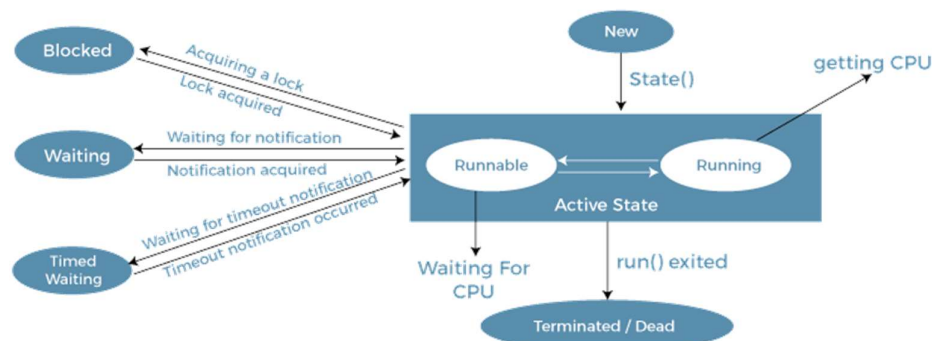


B. Java Thread Methods:

Method	Syntax	Description
start()	t.start();	Starts a new thread.
run()	public void run(){}	Contains code executed by the thread.
sleep(long ms)	Thread.sleep(1000);	Suspends current thread for specified time.
join()	t.join();	Waits until the thread completes.
join(long ms)	t.join(2000);	Waits for a specified time.
yield()	Thread.yield();	Gives chance to other runnable threads.
interrupt()	t.interrupt();	Interrupts a waiting/sleeping thread
isInterrupted()	t.isInterrupted();	Checks if a thread is interrupted.
interrupted()	Thread.interrupted();	Tests and clears interrupt status of current thread.
isAlive()	t.isAlive();	Checks whether thread is running.
currentThread()	Thread.currentThread();	Returns currently executing thread.
getName()	t.getName();	Returns thread name.
setName()	t.setName("T1");	Sets thread name.
getPriority()	t.getPriority();	Gets thread priority.
setPriority()	t.setPriority(8);	Sets thread priority.
getState()	t.getState();	Returns current thread state.
getId() / threadId()	t.threadId();	Returns unique thread identifier.
isDaemon()	t.isDaemon();	Checks if thread is daemon.
setDaemon()	t.setDaemon(true);	Sets thread as daemon.
activeCount()	Thread.activeCount();	Returns active thread count.
getThreadGroup()	t.getThreadGroup();	Returns thread group.
getStackTrace()	t.getStackTrace();	Returns stack trace of thread.
start()	t.start();	Starts a new thread.

C. Life Cycle of a Thread in Java Multithreading:

- In Java, a thread goes through different states from its creation to its termination. To manage the threads in multithreading, it is important to understand the life cycle of a thread (or, a flow of the threads). The main states of a thread are:
 1. **New (Created):** A thread is created but not yet started. It enters this state when you create an object of the Thread class.
 2. **Runnable:** After calling the start() method, the thread becomes ready to run and waits for CPU scheduling. It is now eligible to run but may not be running immediately.
 3. **Running:** When the thread scheduler picks the thread from the runnable pool, it enters the running state and executes its run() method.
 4. **Waiting/Blocked:** A thread can enter a waiting state if it is waiting for another thread to perform a task or release a resource.
 5. **Timed Waiting:** A thread can wait for a specified period using methods like sleep(milliseconds) or join(milliseconds).
 6. **Terminated (Dead):** Once the thread completes its execution or is stopped, it enters the terminated state. A terminated thread cannot be restarted.



- **Transitions Between Thread States**

1. **NEW → start() → RUNNABLE:** A thread moves from the new state to the runnable state when its start() method is called.
2. **RUNNABLE → scheduler picks → RUNNING:** The thread enters the running state when the JVM thread scheduler selects it for execution.
3. **RUNNING → wait()/join() → WAITING:** A running thread moves to the waiting state when it calls wait() or join() without a timeout.
4. **RUNNING → sleep()/join(timeout) → TIMED_WAITING:** A thread enters timed waiting if it calls sleep() or join() with a specified timeout.
5. **RUNNING → BLOCKED → trying to acquire a lock:** A running thread becomes blocked when it attempts to access a synchronized block or method already locked by another thread.
6. **Any state → finished/error → TERMINATED:** A thread enters the terminated state when it completes execution or terminates due to an exception.

- **D. Advantages of Multithreading in Java:**

1. **Improved Performance:** Multiple tasks can run simultaneously, reducing execution time.
2. **Efficient CPU Utilization:** Threads keep the CPU busy by running tasks in parallel.
3. **Responsiveness:** Applications (like GUIs) remain responsive while performing background tasks.
4. **Resource Sharing:** Threads within the same process share memory and resources, avoiding duplication.
5. **Better User Experience:** Smooth execution of tasks like file downloads, animations, and real-time updates.

- **E. How to Achieve Multithreading in Java?**

- We can achieve multithreading in Java in three ways:
 - By extending Thread class
 - By Implementing the Runnable Interface
 - By Implementing the Callable Interface

Method	Syntax	Return Value	Best Use Case
Extending Thread Class	class MyThread extends Thread	No	Simple thread creation
Implementing Runnable Interface	class MyTask implements Runnable	No	Preferred approach, supports inheritance
Implementing Callable Interface	class MyTask implements Callable<T>	Yes	When thread needs to return a result

1. **Extending Thread Class**

- You can create a thread by extending the Thread class and overriding its run() method.
- The run() method contains the code that defines the thread's task. After creating an object of the subclass, you start the thread using the start() method.
- **Example:** The following example demonstrates creating a thread by extending the Thread class:

<pre>class MyThread extends Thread { public void run() { for(int i = 1; i <= 5; i++) System.out.println("Thread running: " + i); } } public class ThreadExample { public static void main(String[] args) { MyThread t1 = new MyThread(); t1.start(); } }</pre>	Output: <pre>Thread running: 1 Thread running: 2 Thread running: 3 Thread running: 4 Thread running: 5</pre> Advantage: Simple. Disadvantage: Cannot extend another class because Java supports single inheritance.
--	---

2. Implementing Runnable Interface (Preferred)

- In Java, we can use threads to process time-consuming tasks parallelly and asynchronously. We can use the **Runnable** interface in Java to run a class instance in multiple threads.
- This is another way to create a thread is by implementing the Runnable interface and providing an implementation for the **run()** method.
- You then pass the Runnable object to a Thread object and call **start()**. This approach can be used when your class is already extending another class.
- **Example:** The following example demonstrates creating a thread by implementing Runnable:

<pre>class MyRunnable implements Runnable { public void run() { for(int i = 1; i <= 5; i++) System.out.println("Runnable Thread: " + i); } } public class RunnableExample { public static void main(String[] args) { MyRunnable r = new MyRunnable(); Thread t1 = new Thread(r); t1.start(); } }</pre>	Output: <pre>Thread running: 1 Thread running: 2 Thread running: 3 Thread running: 4 Thread running: 5</pre> In this example, we created a class MyRunnable that implements the Runnable interface and overridden the run() method. When a thread is created with the MyRunnable instance and executed (t1.start()), the run() method of the MyRunnable instance is called.
--	---

- **Advantage:** Can extend another class and share the same Runnable object among multiple threads.
- **Limitations of Runnable Interface in Java**
 - The run() method of the Runnable interface is of type void and cannot return any values.
 - The Runnable interface doesn't allow to throw any checked exceptions (exceptions caught at compile time. Eg: Exception).

3. Implementing Callable Interface

- Callable interface was introduced to overcome the limitation of the Runnable interface.
- The Callable interface in Java has a single method call() that can return any object as result using generics.
- The class whose instance has to be run as a thread should implement the Callable interface and override the call() method.
- The skeleton of the Callable interface is:


```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```
- **Return Value of Callable Interface in Java**
 - Unlike the Runnable, the Callable instance can't be passed as input while creating a thread.
 - Instead, we must wrap it with a future (Eg. FutureTask) and pass it to the Thread class.
 - The Thread returns a future with the result produced by the Callable instance. We can retrieve the result from the future.

```
import java.util.concurrent.*;

class MyTask implements Callable<Integer> {
    public Integer call() {
        return 100;
    }
}

public class Test {
    public static void main(String[] args) throws Exception {
        FutureTask<Integer> ft =
```

```

        new FutureTask<>(new MyTask());
        Thread t = new Thread(ft);
        t.start();
        System.out.println("The Output from Program:"+ft.get());
    }
}

```

Output: The Output from Program:100

- **Advantage:** Returns a value and can throw checked exceptions.

6. Real Time Example: Web Server Handling Multiple Clients

- A web server can handle multiple client requests simultaneously by assigning each request to a separate thread. This ensures that the server remains responsive even when many clients are connected.
- The following code demonstrates multithreading for a web server handling multiple clients:

```

class ClientRequest extends Thread {
    private String clientName;
    ClientRequest(String name) {
        this.clientName = name;
    }
    public void run() {
        System.out.println(clientName + " request processing started.");
        try {
            Thread.sleep(1500); // Simulate request handling
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(clientName + " request processing completed.");
    }
    public static void main(String[] args) {
        ClientRequest c1 = new ClientRequest("Client1");
        ClientRequest c2 = new ClientRequest("Client2");
        c1.start();
        c2.start();
    }
}

```

Output:

```

Client1 request processing started.
Client2 request processing started.
Client1 request processing completed.
Client2 request processing completed.

```

7. References:

- E Balaguruswamy, (2023). Programming with Java: A Primer. 7th. India: McGraw Hill Education
- Herbert Schildt, (2021). Java: The complete reference, 13th edition. McGraw-Hill Education.
- <https://www.tpointtech.com/multithreading-in-java>
- <https://cse.iitkgp.ac.in/~dsamanta/java/ch6.htm>

Experiment No.9

1. **Title:** City Traffic Control using Multithreading.

2. **Problem Statement:**

A city traffic control department wants to simulate the behavior of vehicles moving through different roads to study congestion patterns. To model this, you are required to design a Java program where each road is represented as a separate thread, and every thread continuously prints the flow of vehicles on that road (e.g., "Car entering Road A", "Bike entering Road B"). The simulation should run multiple threads simultaneously, demonstrating real time parallel activity. The system must show how threads start, execute concurrently, and complete, helping the department analyze how traffic behaves across multiple roads at the same time.

3. **Objectives:**

- To understand **Java multithreading** by **modeling** each road as an independent thread.
- To create and **execute multiple threads concurrently** to simulate vehicle movement.
- To demonstrate the **life cycle of threads**—start, execution, and completion.
- To simulate real-time traffic flow across multiple roads.

4. **Outcomes:**

After completing this problem, students will be able to:

- Create and manage **multiple threads** in Java.
- Demonstrate **concurrent execution** using Thread.
- Observe and explain **thread execution** and **completion**.
- Apply **multithreading** concepts to a **real-world traffic simulation** scenario.

5. **Software/Hardware/Other Requirements:**

- Any CPU with Pentium Processor / similar, 256 MB RAM/ more, 1GB HDD / more.
- Operating System – ubuntu/Fedora 64bit OS
- Software: JDK 17 or later, Code Editor

6. **Theory:**

A. **Thread Class:** The Thread class is used to create and manage threads in Java.

• **Example**

```
class Vehicle extends Thread {
    public void run() {System.out.println("Vehicle is moving"); }
}
```

B. **run() Method:** The run() method contains the task that the thread will execute.

• **Example**

```
class Vehicle extends Thread {
    public void run() {System.out.println("Car is moving"); }
}
```

- When the thread starts, the code inside run() executes.

C. **start() Method:** start() creates a new thread and internally calls run().

• **Example**

```
Vehicle v = new Vehicle();
v.start();
```

D. **Difference between start() and run()**

start()	run()
Creates a new thread	Executes like a normal method
Concurrent execution	No concurrency
JVM calls run() internally	Called directly by programmer

```
Vehicle v = new Vehicle();
v.run(); // Normal method call
v.start(); // New thread created
```

E. Thread Life Cycle in Java

- Thread life cycle in Java refers to the various states a thread passes through during its execution, from creation to termination, as defined in the `java.lang.Thread.State` class.
- All thread states are defined as enum constants in the `java.lang.Thread.State` class. At any given point in time, a thread can be in only one state.
- These states represent virtual machine-level thread states and do not necessarily correspond to the underlying operating system thread states.

F. Thread States in Java:

- A thread in Java can be in **6 states** (`Thread.State`):

State	Description
NEW	Thread object is created, but <code>start()</code> has not been called.
RUNNABLE	Thread is ready to run or currently running.
BLOCKED	Waiting to acquire a lock (enter a synchronized block/method).
WAITING	Waiting indefinitely for another thread's action (<code>wait()</code> , <code>join()</code>).
TIMED_WAITING	Waiting for a specified time (<code>sleep()</code> , <code>wait(timeout)</code> , <code>join(timeout)</code>).
TERMINATED	Thread has finished execution or ended due to an error.

- **Thread States in Java with Simple Examples:**

1. NEW State

- A thread is created but `start()` has not been called yet.

```
Thread t = new Thread();
System.out.println(t.getState()); // NEW
```

- **Real-life example:** Student is registered for an exam but has not entered the exam hall.

2. RUNNABLE State

- After calling `start()`, the thread becomes runnable. It may be running or waiting for CPU time.

```
Thread t = new Thread(() -> {
    System.out.println("Running...");
});
t.start();
System.out.println(t.getState()); // RUNNABLE
```

- **Real-life example:** Students are seated in the exam hall and ready to write the paper.

3. BLOCKED State

- A thread is waiting to acquire a lock held by another thread. [docs.oracle.com]

```
class Test {
    synchronized void display() {
        try {
            Thread.sleep(3000);
        } catch (Exception e) {}
    }
}
Test obj = new Test();
Thread t1 = new Thread(() -> obj.display());
Thread t2 = new Thread(() -> obj.display());
t1.start();
t2.start(); // BLOCKED until t1 releases lock
```

- **Real-life example:** One person is using the printer, another is waiting for it.

4. WAITING State

- A thread waits indefinitely for another thread to complete an action. Commonly caused by `join()` or `wait()`.

```
Thread child = new Thread(() -> {
    System.out.println("Child Thread");
});
child.start();
child.join(); // main thread enters WAITING state
```

- **Real-life example:** Parent waits until the child finishes homework.

5. TIMED_WAITING State

- A thread waits for a specified amount of time. Caused by `sleep()`, `join(timeout)`, etc.

```
Thread t = new Thread() -> {
    try {
        Thread.sleep(5000);
    } catch (Exception e) {}
};
t.start();
```
- During sleep, the thread is in **TIMED_WAITING**.
- Real-life example:** A student takes a 10-minute break and resumes studying after the break.

6. TERMINATED State

- The thread has completed execution.

```
Thread t = new Thread() -> {
    System.out.println("Task Completed");
};
t.start();
t.join();
System.out.println(t.getState()); //TERMINATED
```
- Real-life example:** The exam is over and the student leaves the hall.
- Difference between WAITING and TIMED_WAITING**

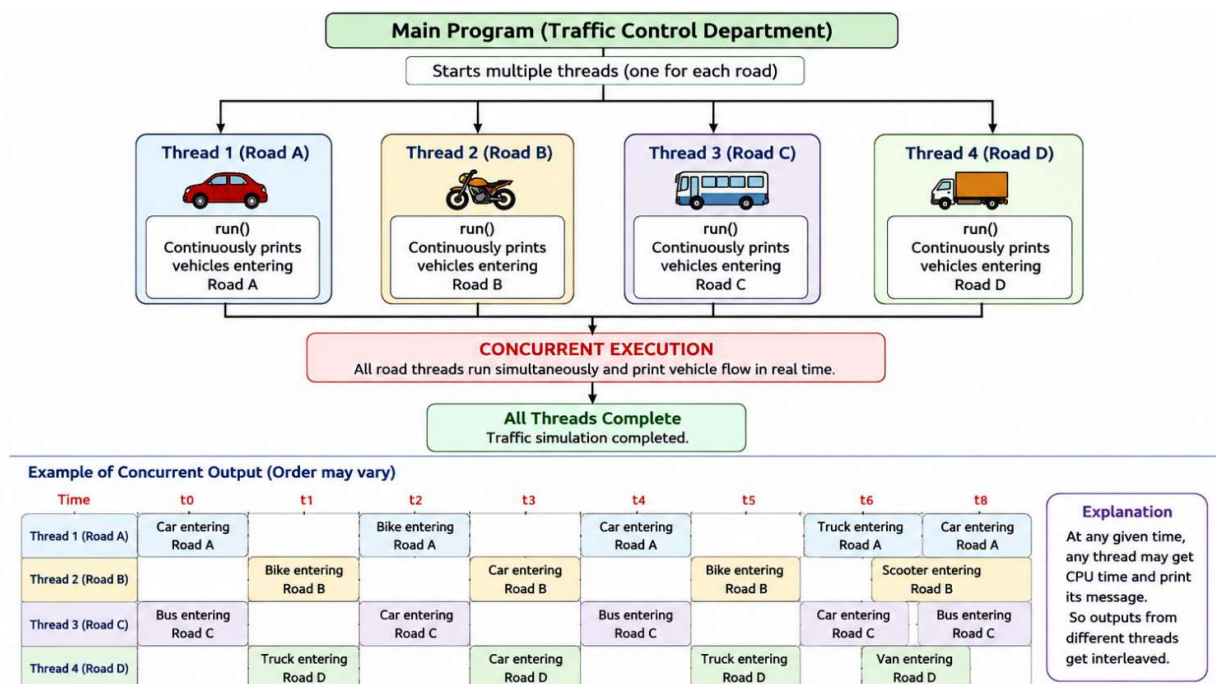
WAITING	TIMED_WAITING
Waits indefinitely	Waits for a fixed time
<code>join()</code> , <code>wait()</code>	<code>sleep()</code> , <code>join(timeout)</code>
Needs notification to resume	Automatically resumes after timeout

G. Multiple Thread Creation

- Multiple Thread Creation means creating and running two or more threads simultaneously. In the traffic simulation, each road can be represented by a separate thread.

H. Concurrent Execution

- All threads execute independently and share CPU time.
- Concurrent execution means multiple threads make progress during the same time period.
- The CPU switches quickly between threads so that all threads appear to run at the same time.
- Each road(thread) prints its vehicle entries independently of others.



I. Thread.sleep() Method

- The Thread.sleep() method is used to pause the execution of the current thread for a specified amount of time.
- During this period, the thread does not consume CPU resources and moves to the timed waiting state. After the specified time elapses, the thread becomes runnable again and waits for the scheduler to pick it for execution.
- **Syntax**
public static void sleep(long mls) throws InterruptedException
public static void sleep(long mls, int n) throws InterruptedException
- **Here,**
 - The version with **one parameter** (millis) is a **native method**; the version with **two parameters** (millis, nanos) is implemented in Java.
 - Since it throws a **checked exception**, you must use a **try-catch block** or declare throws InterruptedException.
 - Any thread, including the **main thread**, can call Thread.sleep().
- **Parameters of Thread.sleep()**
 - **millis (mls):** Specifies the time in **milliseconds** for which the thread will sleep.
 - **nanos (n):** Specifies additional **nanoseconds** (0 to 999,999) to pause the thread for more precise timing.
- **Return Type:** The sleep() method does not return any value.

Examples of Thread.sleep() Method

The following are examples showing different use cases of the **Thread.sleep()** method.

- **Example for Simple Sleep on Main Thread**
- The following example demonstrates a simple pause of 2 seconds in the main thread using Thread.sleep().

<pre>public class SleepExample { public static void main(String[] args) { System.out.println("Start"); try { Thread.sleep(2000); // Pause for 2000 milliseconds (2 seconds) } catch (InterruptedException e) { System.out.println("Thread interrupted"); } System.out.println("End"); } }</pre>	Output: Start End
---	--------------------------------

In this example, the main thread prints "Start", pauses for 2 seconds, and then prints "End".

J. Thread.join() Method

- The join() method is provided by the **java.lang.Thread** class. It allows the current executing thread to wait until the thread on which join() is called finishes its execution.
- In simple terms: One thread waits for another thread to complete before moving forward.
- **How join() Method Works?**
 - Suppose **th** is a thread object that is currently executing.
 - When **th.join()** is called, the calling thread enters the waiting state.
 - The calling thread resumes execution only after **th** has completed its execution (reaches the dead state).
- **Why Use join() Method?**
 - A task must complete before another task starts
 - Thread execution order matters
 - You want to avoid race conditions caused by parallel execution

- It has a **void** type and throws **InterruptedException**. Joining threads in Java has three functions namely viz.,

Method	Description
join()	Waits for this thread to die
join(long millis)	Waits at most millis milliseconds for this thread to die
join(long millis, int nanos)	Waits at most millis milliseconds plus nano nanoseconds for this thread to die

K. Handling InterruptedException

- Why Exception Occurs? If another thread interrupts the sleeping/waiting thread.
- The following example shows how to handle **InterruptedException** when a sleeping thread is interrupted by another thread.

```

public class InterruptHandling {
    public static void main(String[] args) {
        Thread thread = new Thread(() -> {
            try {
                System.out.println("Thread will sleep for 5 seconds.");
                Thread.sleep(5000);
                System.out.println("Thread woke up after sleep.");
            } catch (InterruptedException e) {
                System.out.println("Thread was interrupted during sleep.");
            }
        });
        thread.start();
        try {
            Thread.sleep(2000); // Main thread sleeps for 2 seconds
            thread.interrupt(); // Interrupt the sleeping thread
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

Output:

```

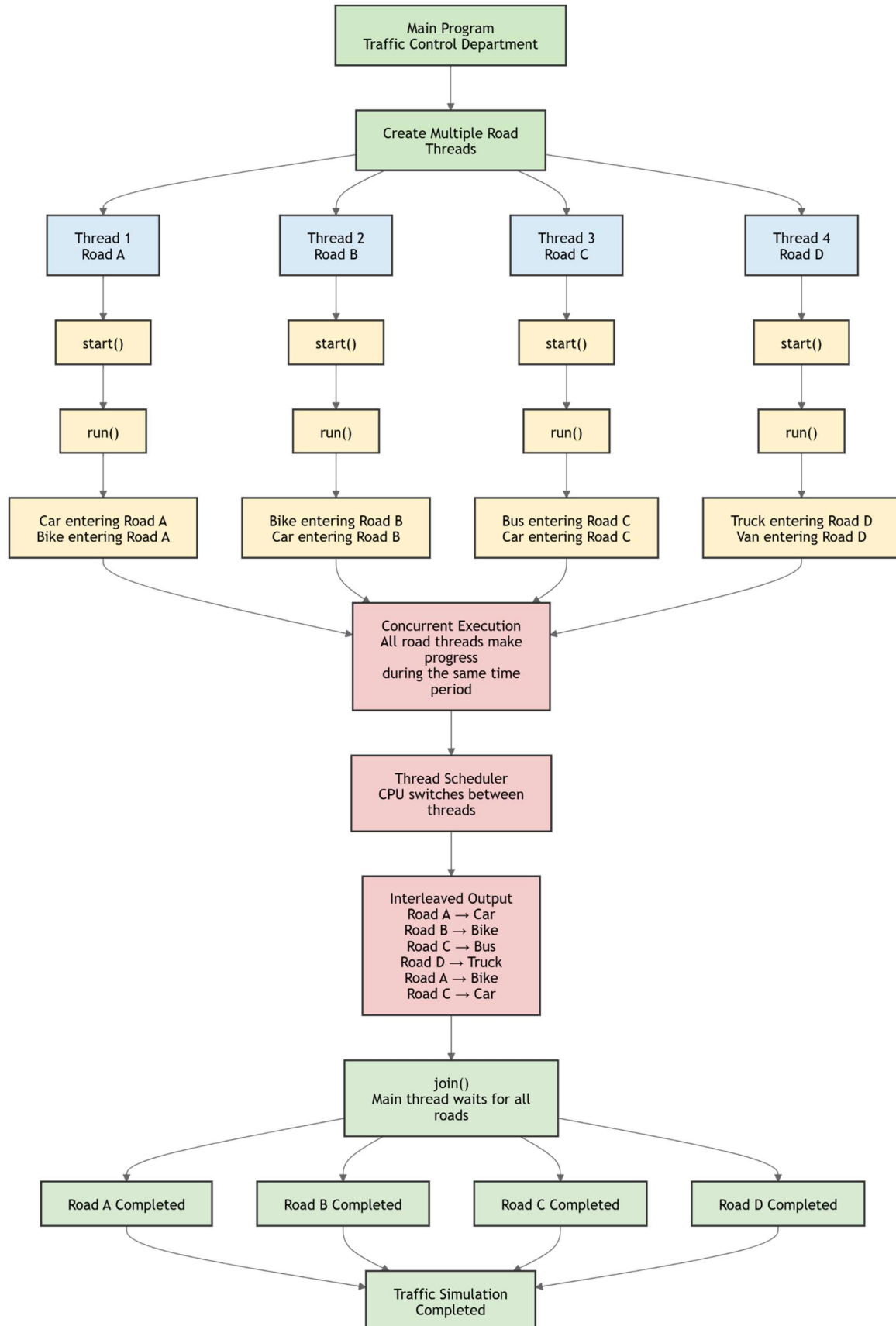
Thread will sleep for 5 seconds.
Thread was interrupted during sleep

```

In this code, the child thread sleeps for 5 seconds but gets interrupted by the main thread after 2 seconds. The catch block handles the interruption gracefully.

L. In short Problem statement

- The Traffic Control Department acts as the main program that creates and starts multiple threads, with each thread representing a different road. For example, Thread 1 represents Road A, Thread 2 represents Road B, Thread 3 represents Road C, and Thread 4 represents Road D. Each thread executes its own `run()` method, where vehicle movement is simulated by repeatedly displaying messages such as "Car entering Road A", "Bike entering Road B", or "Bus entering Road C". The threads can also use `sleep()` to introduce small delays, making the simulation appear like real-time traffic flow.
- Once all road threads are started using the `start()` method, they execute concurrently, meaning vehicle activity on different roads can occur at the same time. The output from the threads may appear in different orders because the Java thread scheduler independently manages their execution. This demonstrates how multiple threads can perform different tasks simultaneously, similar to real traffic conditions where vehicles move on several roads at once. After each thread finishes its assigned simulation, the program can use `join()` to wait for all roads to complete before displaying "Traffic simulation completed."

7. Flowchart Diagram:

8. Implementation:**Program for implementation of Traffic Control Department**

```
// Road class representing a road as a separate thread
class Road extends Thread {
    private String roadName;
    private String[] vehicles;

    public Road(String roadName, String[] vehicles) {
        this.roadName = roadName;
        this.vehicles = vehicles;
    }
    @Override
    public void run() {
        try {
            for (String vehicle : vehicles) {
                System.out.println(vehicle + " entering " + roadName);
                Thread.sleep(1000); // Simulate time gap between vehicles
            }
            System.out.println(roadName + " traffic completed.");
        } catch (InterruptedException e) {
            System.out.println(roadName + " interrupted.");
        }
    }
}

// Main class
public class TrafficSimulation {
    public static void main(String[] args) {
        String[] roadAVehicles = {"Car", "Bike", "Bus", "Truck"};
        String[] roadB Vehicles = {"Bike", "Car", "Auto", "Bus"};
        String[] roadC Vehicles = {"Truck", "Car", "Bike", "Van"};
        Road roadA = new Road("Road A", roadAVehicles);
        Road roadB = new Road("Road B", roadB Vehicles);
        Road roadC = new Road("Road C", roadC Vehicles);
        System.out.println("Starting Traffic Simulation...\n");
        // Start all road threads simultaneously
        roadA.start();
        roadB.start();
        roadC.start();
        try {
            // Wait for all threads to finish
            roadA.join();
            roadB.join();
            roadC.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("\nTraffic Simulation Completed.");
    }
}
```

Output:

```
Starting Traffic Simulation...
Truck entering Road C
Bike entering Road B
Car entering Road A
Car entering Road C
Car entering Road B
Bike entering Road A
Bike entering Road C
Auto entering Road B
Bus entering Road A
Van entering Road C
Bus entering Road B
Truck entering Road A
Road C traffic completed.
Road B traffic completed.
Road A traffic completed.
Traffic Simulation Completed.
```

9. Conclusion:

This experiment demonstrated how synchronization ensures safe and consistent access to shared resources in a multithreaded environment by preventing race conditions. It also highlighted the importance of thread safety, enabling multiple threads to execute concurrently while maintaining data integrity and correct program behavior.

10. Review Questions & Exercises:**a. Fill in the Blanks**

Q. No.	Question	BT Level	CO	PO & PSO
i	The _____() method is used to start a new thread.	BT1	CO2	PO1, PSO1
ii	The _____() method is used to make the main thread wait until another thread finishes.	BT1	CO2	PO1, PSO1
iii	The Java _____ decides which thread gets CPU time for execution.	BT1	CO2	PO1, PSO1
iv	A thread that has finished executing its run() method reaches the _____ state.	BT2	CO2	PO1,2, PSO1

b. Answer the following:

Q. No.	Question	BT Level	CO	PO & PSO
i	Differentiate between Process & Thread.	BT2	CO2	PO1, 2, PSO1
ii	Why should start() be used instead of directly calling run() to achieve multithreading?	BT2	CO2	PO1, 2, PSO1
iii	Explain the role of the Java thread scheduler in concurrent execution.	BT3	CO2	PO1, 2,5, PSO1

c. Programming Assignments:

Q. No.	Question	BT Level	CO	PO & PSO
i	Smart Hospital Emergency Monitoring: A hospital wants to monitor activities occurring in different departments simultaneously. Design a Java program where each department (Emergency, ICU, OPD, Pharmacy) is represented by a separate thread. Each thread continuously displays activities such as patient admissions, medicine dispensing, or emergency alerts. The system should demonstrate concurrent execution and coordinated monitoring of multiple hospital departments.	BT3	CO2	PO1,3,5, PSO1
ii.	University Examination Evaluation System: A university wants to simulate answer sheet evaluation across multiple departments. Design a Java application where each department is represented as a separate thread. Every thread continuously evaluates answer sheets and displays progress updates. The simulation should run all department threads simultaneously to demonstrate parallel evaluation processes.	BT3	CO2	PO1,3,5, PSO1

11. References:

- E Balaguruswamy, (2023). Programming with Java: A Primer. 7th. India: McGraw Hill Education
- Herbert Schildt, (2021). Java: The complete reference, 13th edition. McGraw-Hill Education.
- <https://www.tpointtech.com/join-thread-in-java>
- <https://www.tpointtech.com/thread-sleep-in-java>

Performance Indicators	
Process related (6 Marks)	
1	Logic formation (3)
2	Debugging ability (2)
3	Follow ethical practices (1)
Product related (4 Marks)	
4	Expected output (1)
5	Timely Submission (2)
6	Answer to sample questions (1)

Marks Obtained			Dated signature of Teacher
Process Related (6)	Product Related (4)	Total (10)	